CANDIDATE NUMBER – 25188871
CASA0007 Quantitative Methods – Article

Gentle Density, Rough Reality

Quantitative Analysis of Additional Dwelling Unit Permits in Toronto, Canada

Additional dwelling units (ADUs) are not always 'the low-hanging fruit' in addressing the housing crisis. A quantitative look at ADU permits between 2022 and 2025 in Toronto hints that ADU growth is based on momentum and spillovers, while there are barriers preventing its uptake elsewhere.

As the name suggests, ADUs are dwelling units that exist *in addition* to the primary residence on a residential lot, making them usually smaller. ADUs promise to deliver more housing stock for cheap (relative to new apartment complexes) without radically changing the built character of a neighbourhood, hence the term 'gentle density' that is often associated with ADU developments. Additionally, it offers homeowners an opportunity to monetise their asset by converting parts of their property into dwellings for rent. This would make ADUs an easy win for everyone, especially considering the dual crisis of cost-of-living and housing unaffordability in Canada that exacerbated post-pandemic. Legislation permitting ADUs picked up steam in Toronto from late 2017 onwards, following earlier adopters like California.

In Toronto, ADUs consist of two main typologies. The first type is known as 'secondary suites', which are ADUs that are attached to the primary residence, as shown in Figure 1. The second type is known as 'laneway / garden / rear-yard suites'. Basically, these are ADUs that are detached from the primary residence but still on the same lot (Figure 2). Since

2025, Toronto has also allowed for multiplex conversions up to sixplexes on one lot in some areas but permit data on this is not readily available yet.

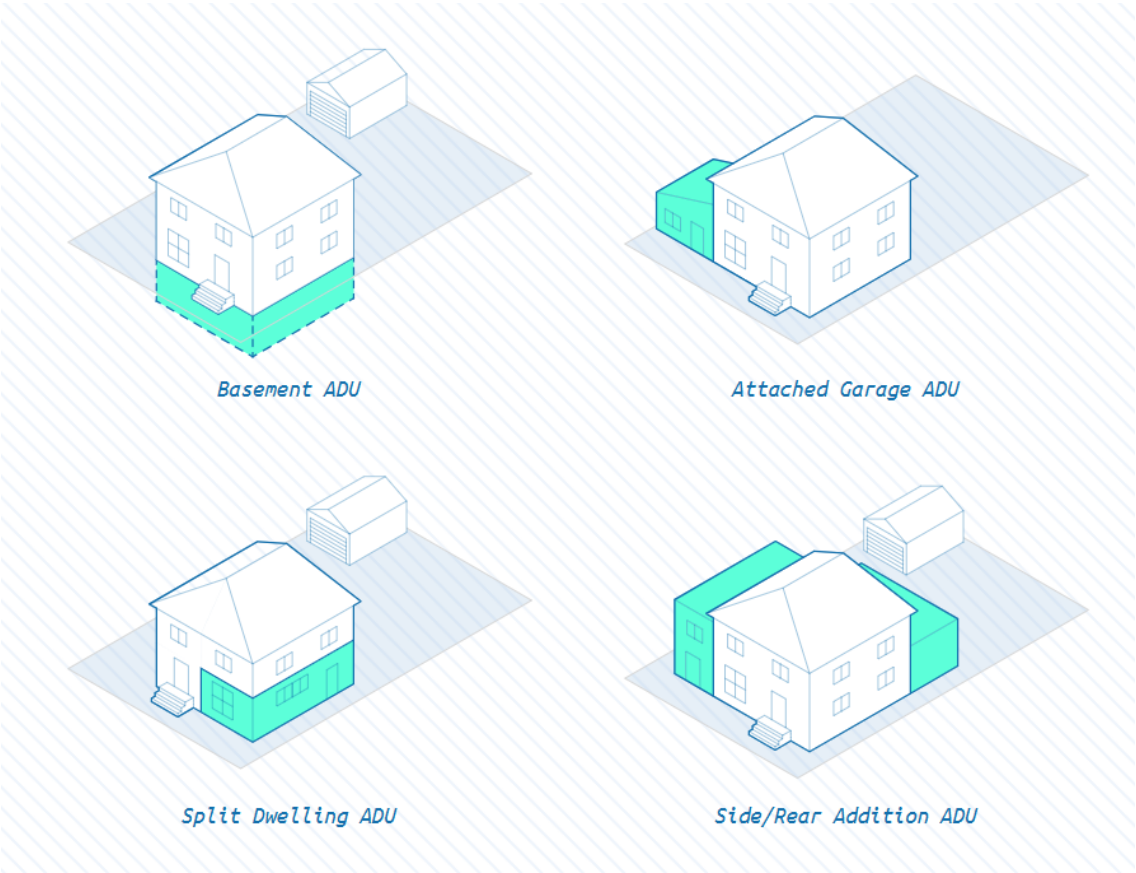


Figure 1: Typologies of 'secondary suites' (attached ADUs) that are common in Toronto (Credits: School of Cities)

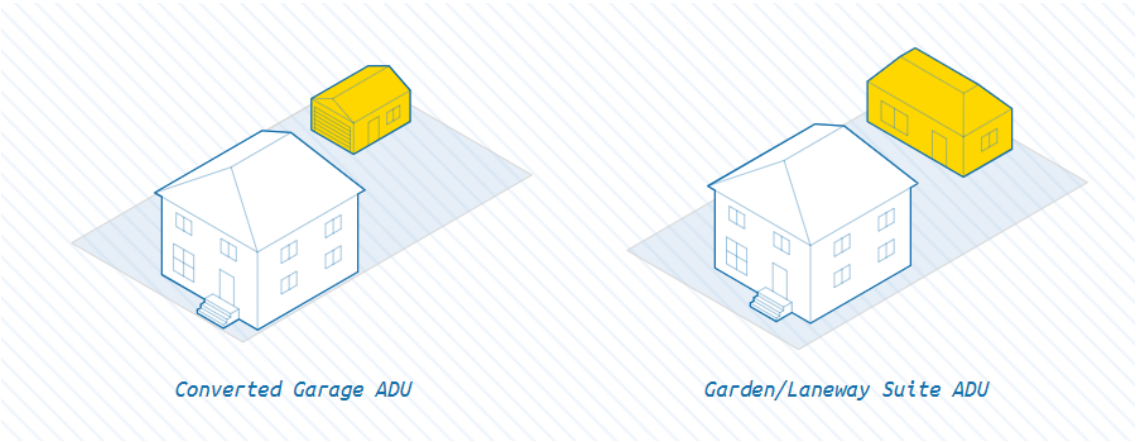


Figure 2: Typologies of 'laneway / garden / rear-yard suites' (detached ADUs) that are common in Toronto (Credits: School of Cities)

Focusing on both active and completed ADU permits that were issued between 2022 and 2025, it shows that permit activity is steadily rising (Figure 3), continuing the trend from before COVID-19. However, they show clustering in some areas, with the densest cluster of 562 permits located west of Downtown Toronto, stretching all the way to High Park and Lawrence Avenue (Figure 4). Although there is a larger cluster of 643 permits, it spans a much larger area from the Don River all the way to Scarborough. Conspicuously, there is little ADU permit activity, and therefore no clusters identified, in Midtown Toronto.

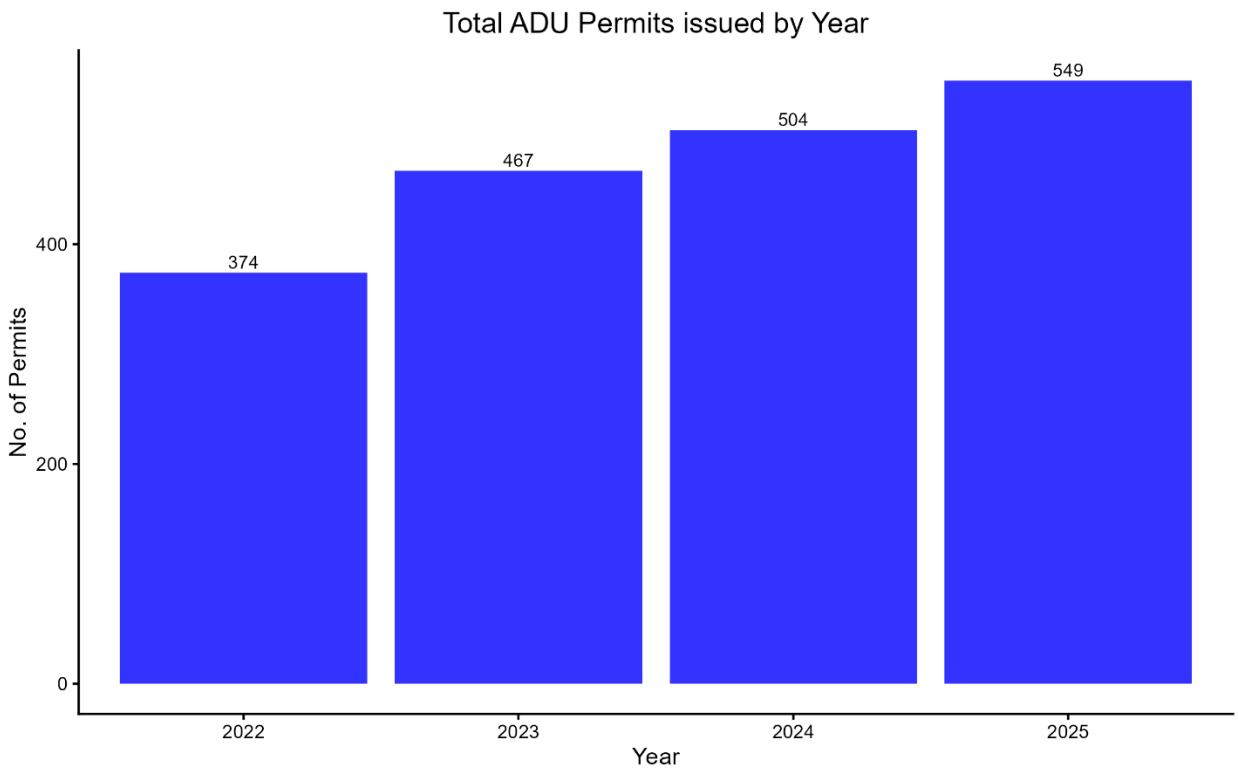


Figure 3: Bar chart of Total ADU Permits issued by the City of Toronto per year, 2022-2025

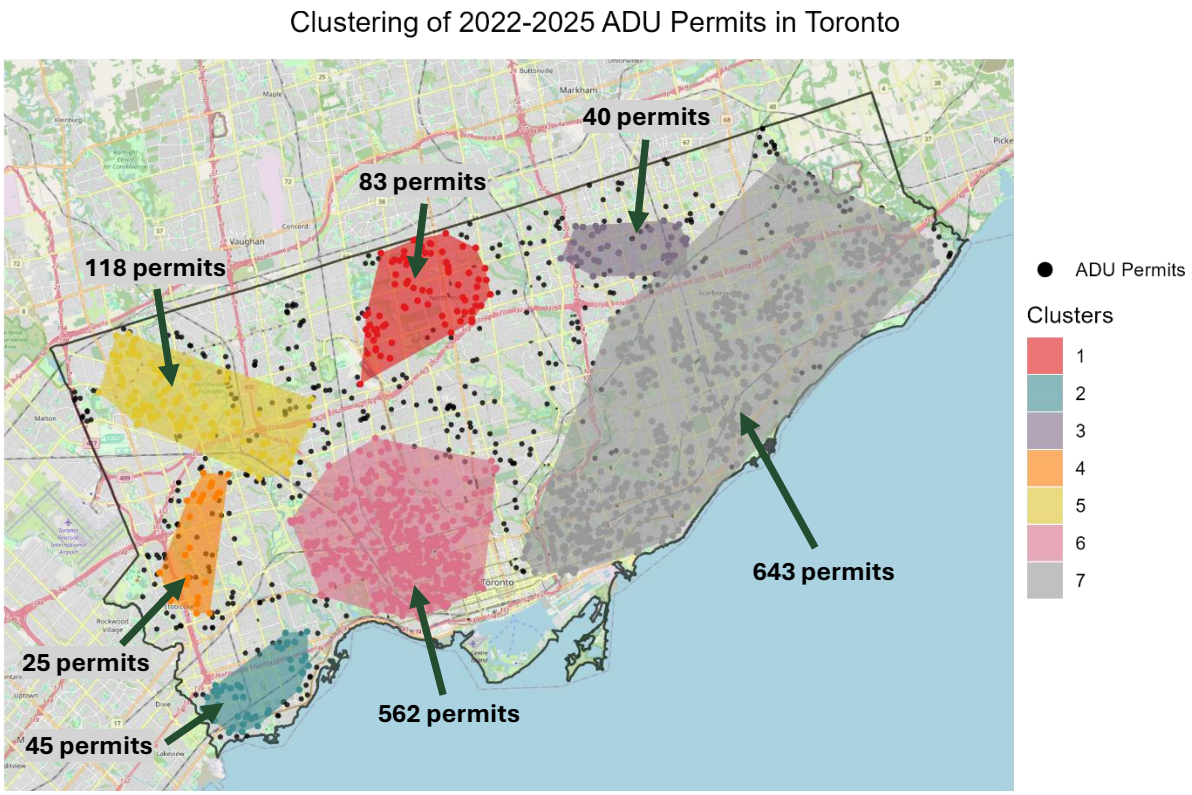


Figure 4: Map of 2022-2025 ADU Permit Clusters identified using HDBSCAN Analysis (minimum point = 19), with count of permits per cluster

Therefore, what could explain this high clustering of ADU activity in Toronto? **What factors may be associated with ADU rates[1] in Toronto Neighbourhoods?**

Through simple regression[2], it is found that a neighbourhood's non-car accessibility, housing typology mix, dwellings' construction period as well as its rates of young

[1] ADU rates are calculated based on proportion of ADUs out of the total housing stock in each Toronto Neighbourhood that can have ADUs (single-detached, semi-detached, row houses and duplexes).

[2] OLS regression modelling between 13 predictors (that can roughly be classified into four categories – Accessibility, Building Stock Characteristics, Neighbourhood Demographics and Housing Pressures) and log-transformed ADU rates [log1p(adu_rate)] was conducted. Some of the predictors (accessibility measures, housing typologies, periods of construction, census family types and census family size) were further compressed using Principal Components Analysis. The modelling was built in an iterative manner to maximise adjusted R-square values while adhering to the five assumptions of OLS regression. See Table 1 and Technical Appendix.

dependents and single parents explain the greatest amount of variance in log-transformed ADU rates (Table 1). However, when controlling for influence from surrounding neighbourhoods[3], the relationship between non-car accessibility and log-transformed ADU rates become insignificant (Table 2).

What this means is that neighbourhoods with high ADU rates are also neighbourhoods that have:

- More single-detached and duplex homes
- More homes built before 1960
- Lower proportions of young dependents
- Higher proportions of single parent households
- Surrounding neighbourhoods with similarly high ADU rates

While subject to further analysis, these patterns could reflect limited familiarity of homeowners, builders and/or planners in Toronto when initiating, planning and/or approving ADU projects. This is likely the case when looking at detached ADUs. Prior to 2022, such ADUs must be adjacent to laneways, which are more common in older neighbourhoods. This is where the 'laneway suite' label came from. After 2022, that restriction was removed and detached ADUs could be built anywhere on the same lot, thus the additional labels of 'garden / rear-yard suite'. This should make detached ADUs possible even in newer neighbourhoods. However, by 2024, only six detached ADUs of the 'garden / rear-yard' variant have been built, suggesting that the increase over the years is mostly powered by secondary and laneway suites.

[3] Spatial lag modelling was conducted based on the set of predictors used in the final OLS regression model, and the influence is split between the how the predictor affects log-transformed ADU rates just in the neighbourhood and for surrounding neighbourhoods as well. See Table 2 and Technical Appendix.

Additionally, they could imply that households with fewer young dependents could spare more space for ADU conversions. Meanwhile, the promise of additional rental income from ADUs, coupled with the likelihood that single parent households may rely more on community childcare facilities (thus requiring less living space), may explain the positive non-linear relationship between single parents and ADU rates.

These patterns differ from Californian cities, where it was found that ADUs were happening in 'diverse, transit-accessible neighbourhoods where... homeowners... still have a mortgage'. This is because rates of visible minority or owners with mortgages did not record significant relationships with ADU rates across Toronto Neighbourhoods (Table 1). Perhaps, among other things, a cancelling-out effect is going on where despite the benefits, there are structural barriers that constrain the uptake of ADUs by homeowners, especially visible minority homeowners, which is also a concern in California.

When looked together with the cluster map, the patterns may suggest a causal pathway where surrounding neighbourhood's activity is the factor that influences a neighbourhood's ADU rates, moderated in clustering intensity by non-car accessibility. The largest cluster in the east with 643 permits spans across a transit desert that is served by just one subway line. Meanwhile, the densest cluster with 562 permits is in some of the most transit-accessible parts of Toronto, with two subway lines and numerous streetcar routes currently serving the area. Therefore, even though there is a lot of ADU permit activity going on in the east that are near enough to each other to be identified as one cluster, they are not as densely located as the ones in downtown. This means the growth of ADUs in Toronto could be due to diffusion, learning-by-seeing, or other unmeasured-but-spatially-patterned factors, which intensifies when there is good transit accessibility. Non-car accessibility alone cannot be taken as a factor because Midtown Toronto is also well served by public transit, but no clusters are identified there.

To an extent, the City may have acknowledged some of these patterns and are working to address them. On top of approving for ADUs of up to sixplexes on one lot, the City has also published pre-approved ADU plans that homeowners and builders could easily adopt, and has undertaken demonstration projects to showcase the models. However, these projects are in areas where clusters have been identified, potentially further localising ADU permit activity in the future.

Table 1: Summary of OLS Regressions of predictors on log-transformed ADU rates [log1p(adu_rate)], with the estimate and p-value of each factor

	Model 1	Model 2	Model 3	Model 4	Model 5	Model 6	Model 7	Model 8
Constant	0.322 *** (0.000)	0.322 *** (0.000)	0.322 *** (0.000)	0.449 (0.530)	0.516 *** (0.00)	-1.184 (0.668)	-4.190 *** (0.000)	-0.072 (0.645)
Non-Car Accessibility (PC)	0.032 . (0.100)	0.083 *** (0.000)	0.080 *** (0.000)	0.045 . (0.090)	0.049 * (0.048)	0.068 ** (0.010)	0.081 *** (0.000)	0.068 ** (0.004)
Single-Det / Duplex (PC)		0.114 *** (0.000)	0.114 *** (0.000)	0.113 *** (0.000)	0.116 *** (0.000)	0.089 *** (0.000)	0.103 *** (0.000)	0.122 *** (0.000)
Semi-Det/Row vs Single-Det (PC)		0.028 . (0.079)	0.030 . (0.055)	0.024 (0.164)				
Row vs Semi-Det (PC		0.011 (0.521)						
Before-60s Period (PC)		0.061 ** (0.001)	0.056 *** (0.001)	0.098 *** (0.000)	0.076 *** (0.000)	0.082 *** (0.000)	0.081 *** (0.000)	0.091 *** (0.000)
60s-80s Period (PC)		0.006 (0.710)						
% of Older Dependents				-0.001 (0.813)				
% of Younger Dependents				-0.018 . (0.058)	-0.014 * (0.029)	-0.030 ** (0.005)	-0.033 *** (0.000)	-0.022 *** (0.000)
% of Movers from within Canada, past five yrs (LT)				-0.037 (0.777)				
% of Movers from Overseas, past five years (LT)				-0.001 (0.981)				
% of Visible Minority (LT)				0.070 (0.371)				
2-person vs Larger Families (PC)						0.205 ** (0.005)	0.192 ** (0.006)	
% of Couples-with-Children Household Type (LT)						0.991 *** (0.001)	0.976 *** (0.000)	
% of Single Parent Household Type (LT)						0.416 ** (0.001)	0.489 *** (0.000)	0.237 *** (0.000)
% of Owners with Mortgages						0.002 (0.346)		
% of Owners with Unaffordable Mortgage (LT)						-0.063 (0.515)		
% of Households in Unsuitable Housing (LT)						-0.100 . (0.077)	-0.048 (0.258)	
2019 Median After-Tax Income (LT)						-0.256 (0.156)		
GINI Index (LT)						0.008 (0.941)		
N = 157								
Adj R-squared	0.011	0.294	0.300	0.315	0.305	0.424	0.426	**0.382**

**** p < 0.001; ** p < 0.01; * p < 0.05; . p < 0.1; PC = Predictor generated from a Principal Components Analysis; LT = Predictor is log-transformed*

Table 2: Summary of direct and indirect impacts of predictors on log-transformed ADU rates [log1p(adu_rate)] in from spatial lag modelling based on queen's contiguity rule

	Direct Impact Estimate (SD)	Indirect Impacts Estimate (SD)	Total P-value
Non-Car Accessibility (PC)	0.025 (0.021)	0.044 (0.041)	0.255
Single-Det / Duplex (PC)	0.092 (0.015)	0.158 (0.055)	0.000
Before-60s Period (PC)	0.069 (0.015)	0.119 (0.046)	0.000
% of Younger Dependents	-0.018 (0.006)	-0.030 (0.015)	0.010
% of Single Parent Household Type (LT)	0.146 (0.048)	0.250 (0.113)	0.007
Rho value = 0.675, **p-value =** 0.000			

PC = Predictor generated from a Principal Components Analysis; LT = Predictor is log-transformed

Technical Appendix - Gentle Density, Rough Reality

25188871

2026-01-12

```r
# for loading all sorts of data
library(tidyverse)
library(sf)
library(sp)
library(data.table)
library(here)

# for general data manipulation
library(janitor)

# to make maps
library(tmap)
library(tmaptools)
library(ggplot2)
library(ggside)
library(ggspatial)
library(prettymapr)
library(RColorBrewer)
library(cols4all)

# to do points and cluster analysis
library(spatstat)
library(fpc)
library(dbscan)

# to do spatial autocorrelation analysis
library(spdep)

# to do regressions
library(performance)
library(tidypredict)
library(spatialreg)
library(jtools)
library(lme4)
library(tidymodels)
library(car)

# to do PCA/Clustering, and I know the codes in Python not in R
library(reticulate)
use_condaenv("r-reticulate", required = TRUE)
py_config()

## python:         C:/Users/mkbs_/AppData/Local/r-miniconda/envs/r-
reticulate/python.exe
```

```
## libpython:      C:/Users/mkbs_/AppData/Local/r-miniconda/envs/r-
reticulate/python312.dll
## pythonhome:     C:/Users/mkbs_/AppData/Local/r-miniconda/envs/r-reticulate
## version:        3.12.12 | packaged by conda-forge | (main, Oct 22 2025,
23:13:34) [MSC v.1944 64 bit (AMD64)]
## Architecture:   64bit
## numpy:          C:/Users/mkbs_/AppData/Local/r-miniconda/envs/r-
reticulate/Lib/site-packages/numpy
## numpy_version:  2.4.0
##
## NOTE: Python version was forced by use_python() function
```

Research Qn

1) Where are ADUs likely to happen in Toronto?
2) What are the factors that are associated with the propensity of ADU occurrences in Toronto?

Output and Plan for Analysis

To answer the first research question, a point cluster analysis of ADU permits (2022-2025) will be conducted using HDBSCAN.

To answer the second research question, an OLS Regression will be conducted. The dependent variable will be set as ADU rates. The independent variables are as follows:
- **ACCESSIBILITY**
| Spatial Access Measures, as provided by Statistics Canada (https://www150.statcan.gc.ca/n1/pub/27-26-0001/272600012023001-eng.htm)
- **BUILDING STOCK CHARACTERISTICS**
| Housing Typologies
| Dwellings' Period of Construction
- **NEIGHBOURHOOD DEMOGRAPHICS**
| Young vs Old Dependents
| Visible Minority Rate
| Mover-Migrator Proportions
- **HOUSING PRESSURES**
| Census Family Sizes
| Census Family Types
| Owners with Mortgage Rate
| Mortgage Unaffordability Rate (Owner Households who spend >=30% of Income on Shelter Costs)
| Unsuitable Housing Rate
| 2019 Median After-Tax Income
| GINI Coefficient

With the exception of Spatial Access Measures, all the data is derived from Census 2021 data, which the City of Toronto has aggregated into Toronto Neighbourhood-level detail (https://open.toronto.ca/dataset/neighbourhood-profiles/).

Execution of Analysis

Getting ADU Permit Data

The City of Toronto releases two datasets on Building Permits - one for active permits (https://open.toronto.ca/dataset/building-permits-active-permits/) and another for cleared permits (https://open.toronto.ca/dataset/building-permits-cleared-permits/). The datasets are updated daily, and this analysis uses the version as of January 1, 2026.

Both will be filtered for permits issued on or after January 1, 2022. For active permits, the status **'Permit Issued'** and **'Revision Issued'** will be selected. For cleared permits, the status **'Closed'** will be selected. For both datasets, the work **'New Laneway / Rear Yard Suite'** and **'Second Suite (New)'** will be selected. Each permit comes with an Address ID, which is helpful for geocoding because it can be joined with the Toronto Address Point data (https://open.toronto.ca/dataset/address-points-municipal-toronto-one-address-repository/).

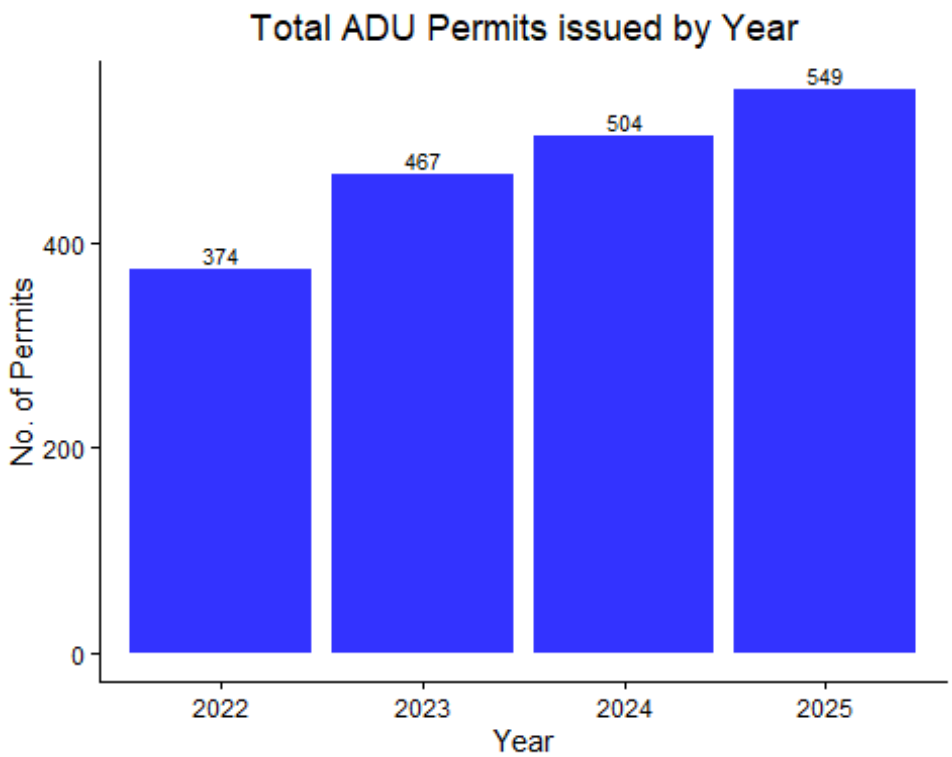
```
# filtering the cleared permit dataset
permit_cleaned <- read.csv('Cleared Building Permits since 2017.csv') %>%
  clean_names() %>%

  # setting the necessary columns as numeric
  mutate(
    issued_date = as.Date(issued_date, format = "%Y-%m-%d"),
    issued_year = as.integer(format(issued_date, "%Y"))
    ) %>%

  # filtering for ADU permits, which permits were issued on or after January
1, 2022 and the final status is Closed (indicating completion)
  filter(work %in% c("New Laneway / Rear Yard Suite", "Second Suite (New)"),
         issued_date > as.Date("2021-12-31"),
         status == 'Closed') %>%

  # since some permits have revisions, they are separately recorded in the
dataset. This is why it is necessary to group_by and summarise
  group_by(permit_num) %>%

  # ensuring that each permit has the geo_id and year of issue
  summarise(
    geo_id = first(geo_id),
    issued_year = first(issued_year)
    )
```

```r
# ---------------------------------------------------------------------------
# now filtering the active permit dataset
permit_active <- read.csv('building-permits-active-permits.csv') %>%
  clean_names() %>%

  # setting the necessary columns as numeric
  mutate(
    issued_date = as.Date(issued_date, format = "%Y-%m-%d"),
    issued_year = as.integer(format(issued_date, "%Y"))
    ) %>%

  # filtering for ADU permits, which permits were issued on or after January
1, 2022 and the final status is either Permit Issued or Revision Issued
  filter(work %in% c("New Laneway / Rear Yard Suite", "Second Suite (New)"),
         issued_date > as.Date("2021-12-31"),
         status %in% c('Permit Issued', 'Revision Issued')) %>%

  # since some permits have revisions, they are separately recorded in the
dataset. This is why it is necessary to group_by and summarise
  group_by(permit_num) %>%

  # ensuring that each permit has the geo_id and year of issue
  summarise(
    geo_id = first(geo_id),
    issued_year = first(issued_year)
    )

# ---------------------------------------------------------------------------
# Now combining both filtered datasets together
permit_total <- bind_rows(permit_cleaned, permit_active) %>%

  # redoing group_by and summarise in order to prevent potential double
counting across the two datasets
  group_by(permit_num) %>%

  # ensuring that each permit has the geo_id and year of issue
  summarise(
    geo_id = first(geo_id),
    issued_year = first(issued_year)
    )

# ---------------------------------------------------------------------------
# joining Address Point Data with the filtered ADU permits data
adu_permits <- permit_total %>%
  left_join(.,
            st_read('Address Points - 4326.shp') %>%
```

```r
              clean_names() %>%
              dplyr::select(address2, geometry),
            by = c('geo_id' = 'address2')) %>%
  st_as_sf(.,
           sf_column_name = 'geometry',
           crs = 4326) %>%
  filter(!st_is_empty(geometry)) %>%

  mutate(issued_year = factor(issued_year, levels = c(2022, 2023, 2024,
2025)))
```

```
## Reading layer `Address Points - 4326' from data source
##   `C:\Users\mkbs_\Documents\UCL\Quant-Methods\Toronto Gentle
Density\Address Points - 4326.shp'
##   using driver `ESRI Shapefile'
## Simple feature collection with 525324 features and 37 fields
## Geometry type: MULTIPOINT
## Dimension:     XY
## Bounding box:  xmin: -79.63824 ymin: 43.58649 xmax: -79.12195 ymax:
43.85415
## Geodetic CRS:  WGS 84
```

```r
# ---------------------------------------------------------------------------
---
# showing histogram of filtered ADU permits data by year
adu_year_counts <- adu_permits %>%
  count(issued_year, name = "n_permits")

p <- ggplot(adu_year_counts, aes(x = factor(issued_year), y = n_permits)) +
  geom_col(alpha = 0.8, fill = "blue") +
  geom_text(aes(label = n_permits), vjust = -0.3, size = 3) +
  labs(title = "Total ADU Permits issued by Year",
       x = "Year",
       y = "No. of Permits") +
  theme_classic() +
  theme(plot.title = element_text(hjust = 0.5))

p
```




```
ggsave("adu_permits_by_year.png", plot = p, width = 8, height = 5, dpi = 300)
```

Also loading the shapefile of Toronto Neighbourhoods (https://open.toronto.ca/dataset/neighbourhoods/) to facilitate analysis later.

```
toronto_neigh <- st_read('Neighbourhoods - 4326.geojson') %>%
  st_transform(., 4326) %>%
  dplyr::select(AREA_SHORT_CODE, geometry)

## Reading layer `Neighbourhoods - 4326' from data source
##   `C:\Users\mkbs_\Documents\UCL\Quant-Methods\Toronto Gentle
Density\Neighbourhoods - 4326.geojson'
##   using driver `GeoJSON'
## Simple feature collection with 158 features and 11 fields
## Geometry type: MULTIPOLYGON
## Dimension:     XY
## Bounding box:  xmin: -79.63926 ymin: 43.581 xmax: -79.11527 ymax: 43.85546
## Geodetic CRS:  WGS 84
```

Creating a DB-Toronto Neighbourhood Relationship

Data on Spatial Access Measures given by Statistics Canada is given at Dissemination Block level. Toronto Neighbourhoods however is a custom geographical unit created by City of Toronto that is based on Statistics Canada's Census Tract.

In theory, we need a relationship file that maps out each DB to Census Tract (which exists, link) and then to Neighbourhood (this is the one that is missing). Thus a messy workaround.

as seen below, needs to be conducted beforehand to establish this relationship. At the end, we need to have an object called **toronto_dgrf** that maps out how all 13,323 DBs in the City of Toronto maps out to the Neighbourhood.

```
# loading the full relationship file
toronto_rs <- read.csv('2021_98260004.csv') %>%

  # selecting the column with IDs of Dissemination Blocks (DB) and their
linked Census Tract (CT) IDs, and then renaming them
  dplyr::select(CTDGUID_SRIDUGD, DBDGUID_IDIDUGD) %>%
  rename(CTUID = CTDGUID_SRIDUGD, DBUID = DBDGUID_IDIDUGD) %>%

  # Dissemination Blocks have a standard number formatting, which are as
follows
  # 2021 refers to the Census Year that the ID is created
  # S refers to the fact that this is a statistical geographical unit
  # 0513 is the code to identify that this is an ID for a Dissemination Block
  # 35 is the province ID for Ontario, where the City of Toronto is
  # 20 is the Census Subdivision ID for the City of Toronto
  filter(str_detect(`DBUID`, '^2021S05133520')) %>%

  # the remaining IDs need to be cleaned to remove the earlier initials
because they are not relevant in the analysis down the road
  mutate(DBUID = str_remove(DBUID, "2021S0513"),
         CTUID = str_remove(CTUID, "2021S0507"))

# ---------------------------------------------------------------------------
---
# we first load the Census Tract file for all of Canada
ct_to_neigh <- st_transform(st_read('lct_000b21a_e.shp', quiet = TRUE),
26917) %>%
  dplyr::select(CTUID, geometry) %>%

  # filter this for CTs in Toronto, based on Toronto DB-CT relationship
generated earlier
  filter(CTUID %in% unique(toronto_rs$CTUID)) %>%

  # this is then intersected with the Toronto Neighbourhoods shapefile
  st_intersection(., st_transform(toronto_neigh, 26917)) %>%

  # calculate the area of each geometry
  mutate(area = st_area(geometry)) %>%

  # since the CT shapefile and Toronto Neighbourhood shapefile are not
produced by the same author, the intersection produces slivers of some CTs
that are split into different Neighbourhoods --> the solution I adopted is to
group the outcomes by the same CT ID and take the one with the biggest area
(because the duplicate smaller areas are due to the slightly imprecise
boundaries between the two shapefiles)
```

```
  group_by(CTUID) %>%
  slice_max(order_by = area, n = 1, with_ties = FALSE) %>%

  # removing geometry because all I need is the relationship between CT and
Neighbourhoods
  ungroup() %>%
  st_drop_geometry() %>%
  dplyr::select(AREA_SHORT_CODE, CTUID)

# ---------------------------------------------------------------------------
---
# now all is needed to join the CT-Neighbourhood relationship to the DB-CT
relationship!
toronto_dgrf <- toronto_rs %>%
  left_join(., ct_to_neigh,
            by = 'CTUID')
```

Getting Spatial Access Measures Data

On top of the fact that Spatial Access Measures data is given at the DB level, it is also provided for VARIOUS modes of transportation to VARIOUS types of destination. The personal decision is made to only bring in the data for Cycling (All Ages and Abilities, to represent the lowest common denominator for cycling), Public Transport (Peak and Off-Peak separately) and Walking, but each mode consists of the mean access score to those various destinations.

While we have the full DB-Neighbourhood relationships, another personal decision has been made to aggregate the data through a simple median to reduce the impact of one highly accessible DB on the whole neighbourhood.

```
# setting up a list of the destinations used across the different transport
mode datasets
# HF refers to healthcare facilities
# EMP refers to employment opportunities
# SRF refers to sports and recreational facilities
# PSEF refers to post-secondary educational facilities
# EF refers to K-12 educational facilities
# CAF refers to culture and arts facilities
# CCF refers to child care facilities
acc_cols_no_gcs = c('acs_idx_hf',
                    'acs_idx_emp',
                    'acs_idx_srf',
                    'acs_idx_psef',
                    'acs_idx_ef',
                    'acs_idx_caf',
                    'acs_idx_ccf')

# ---------------------------------------------------------------------------
---
```

```r
# loading Cycling (All Ages and Abilities) SAM dataset first
cycling <- read.csv('acs_cycling_all_ages_and_abilities.csv', na = '..') %>%

  # filtering for Toronto DBs
  filter(str_detect(`DBUID`, '^3520')) %>%
  mutate(DBUID = as.character(DBUID),

         # calculating the mean access score to the seven destination types
         mean_acc_no_gcs = rowMeans(
           dplyr::select(., acc_cols_no_gcs),
           na.rm = TRUE
         )) %>%

  # the DB-Neighbourhood relationship is then joined to the mean DB cycling
access data
  left_join(., toronto_dgrf, by = 'DBUID') %>%

  # group_by unique Toronto Neighbourhoods, and calculating median cycling
access data for each neighbourhood
  group_by(AREA_SHORT_CODE) %>%
  summarise(median_cycling = median(mean_acc_no_gcs, na.rm = TRUE))

# ---------------------------------------------------------------------------
---
# Repeating the steps above but for Public Transport (Peak)
pt_peak <- read.csv('acs_public_transit_peak.csv', na = '..') %>%
  filter(str_detect(`DBUID`, '^3520')) %>%
  mutate(DBUID = as.character(DBUID),
         mean_acc_no_gcs = rowMeans(
           dplyr::select(., acc_cols_no_gcs),
           na.rm = TRUE
         )) %>%
  left_join(., toronto_dgrf, by = 'DBUID') %>%
  group_by(AREA_SHORT_CODE) %>%
  summarise(median_pt_peak = median(mean_acc_no_gcs, na.rm = TRUE))

# ---------------------------------------------------------------------------
---
# Repeating the steps above but for Public Transport (Off-Peak)
pt_offpeak <- read.csv('acs_public_transit_offpeak.csv', na = '..') %>%
  filter(str_detect(`DBUID`, '^3520')) %>%
  mutate(DBUID = as.character(DBUID),
         mean_acc_no_gcs = rowMeans(
           dplyr::select(., acc_cols_no_gcs),
           na.rm = TRUE
         )) %>%
  left_join(., toronto_dgrf, by = 'DBUID') %>%
  group_by(AREA_SHORT_CODE) %>%
  summarise(median_pt_offpeak = median(mean_acc_no_gcs, na.rm = TRUE))
```

```
# ---------------------------------------------------------------------------
---
# Repeating the steps above but for Walking
walking <- read.csv('acs_walking.csv', na = '..') %>%
  filter(str_detect(`DBUID`, '^3520')) %>%
  mutate(DBUID = as.character(DBUID),
         mean_acc_no_gcs = rowMeans(
           dplyr::select(., acc_cols_no_gcs),
           na.rm = TRUE
         )) %>%
  left_join(., toronto_dgrf, by = 'DBUID') %>%
  group_by(AREA_SHORT_CODE) %>%
  summarise(median_walking = median(mean_acc_no_gcs, na.rm = TRUE))
```

Getting Census Data

Census 2021 Data has been helpfully aggregated into Toronto Neighbourhood-level detail by the City of Toronto, sparing us of the nightmare of aggregating. Thus the challenge of this data cleaning section is to identify the relevant sets of data, and, where applicable, converting the counts data into percentage data.

```
# identifying relevant information needed from the Census 2021 data
relevant_col_id = c(1,
                    30, 32, # broad age distribution percentages
                    144, 214,  # median after tax 2019, GINI after tax
                    216, # occupied private dwelling TOTAL, to be used as
denominator
                    217, 218, 219, 220,  # housing types
                    324, # unsuitable housing count
                    326, 327, 328, 329, 330, 331, 332, 333,  # period of
const
                    368, 369, # mortgage unaffordability share
                    385, # census families TOTAL, to be used as denominator
                    386, 387, 388, # census family sizes
                    394, 397, 400, 401, # census family types
                    1641, 1642, # visible minority
                    1941, 1942, 1944, 1947, 1948, 1949) # migrator types

# ---------------------------------------------------------------------------
---
# Loading Census 2021 data, filtering for the relevant information, and
transposing it
census_2021 <- fread('neighbourhood-profiles-2021-158-model.csv') %>%
  mutate(row_id = row_number()) %>%
  filter(row_id %in% relevant_col_id)

census_transposed <- census_2021 %>%
  dplyr::select(-row_id) %>%
```

```
  mutate(across(1, make.unique)) %>%
  column_to_rownames(var = names(.)[1]) %>%
  t() %>%
  as.data.frame() %>%
  rownames_to_column("AREA_NAME") %>%
  filter(AREA_NAME != 'row_id') %>%
  clean_names()

# ---------------------------------------------------------------------------
---
# Creating the Rates from the Count Data in the Census
census_cleaned <- census_transposed %>%

  mutate(
    # reformatting neighbourhood number to have three characters (001 to 174)
    neighbourhood_number = as.character(sprintf("%03d",
as.integer(census_transposed$neighbourhood_number))),

    # initial reformatting of every other cols except area name and area code
as numeric, to facilitate rates calculation
    across(3:ncol(.), ~ as.numeric(.)),

    # calculating housing type shares for each neighbourhood
    single_det_rate = single_detached_house /
total_occupied_private_dwellings_by_structural_type_of_dwelling_25_percent_sa
mple_data,
    semi_det_rate = semi_detached_house /
total_occupied_private_dwellings_by_structural_type_of_dwelling_25_percent_sa
mple_data,
    row_hse_rate = row_house /
total_occupied_private_dwellings_by_structural_type_of_dwelling_25_percent_sa
mple_data,
    duplex_rate = apartment_or_flat_in_a_duplex /
total_occupied_private_dwellings_by_structural_type_of_dwelling_25_percent_sa
mple_data,
    eligible_type_count = single_detached_house + semi_detached_house +
row_house + apartment_or_flat_in_a_duplex,

    # calculating housing unsuitability rate for each neighbourhood
    unsuitability_rate = (not_suitable /
total_occupied_private_dwellings_by_structural_type_of_dwelling_25_percent_sa
mple_data)*100,

    # calculating construction period shares for each neighbourhood
    bef60_hse_rate =
x1960_or_before/total_occupied_private_dwellings_by_structural_type_of_dwelli
ng_25_percent_sample_data,
    x60s70s_hse_rate =
x1961_to_1980/total_occupied_private_dwellings_by_structural_type_of_dwelling
```

```
_25_percent_sample_data,
    x80s_hse_rate =
x1981_to_1990/total_occupied_private_dwellings_by_structural_type_of_dwelling
_25_percent_sample_data,
    x90s_hse_rate =
x1991_to_2000/total_occupied_private_dwellings_by_structural_type_of_dwelling
_25_percent_sample_data,
    x00s_hse_rate = (x2001_to_2005 +
x2006_to_2010)/total_occupied_private_dwellings_by_structural_type_of_dwellin
g_25_percent_sample_data,

    # calculating rates of census family sizes for each neighbourhood
    x2p_rate = x2_persons /
total_census_families_in_private_households_by_family_size_25_percent_sample_
data,
    x3p_rate = x3_persons /
total_census_families_in_private_households_by_family_size_25_percent_sample_
data,
    x4p_rate = x4_persons /
total_census_families_in_private_households_by_family_size_25_percent_sample_
data,

    # calculating rates of census family types for each neighbourhood
    couples_wchildren = ((with_children + with_children_1) /
total_census_families_in_private_households_by_family_size_25_percent_sample_
data)*100,
    single_parent = ((in_which_the_parent_is_a_woman +
in_which_the_parent_is_a_man) /
total_census_families_in_private_households_by_family_size_25_percent_sample_
data)*100,

    # calculating visible minority rates for each neighbourhood
    vis_min_rate = (total_visible_minority_population /
total_visible_minority_for_the_population_in_private_households_25_percent_sa
mple_data)*100,

    # calculating mover (moving in from within the city) and migrant (moving
in from outside the city) rates for each neighbourhood
    domestic_movers = ((non_migrants + intraprovincial_migrants +
interprovincial_migrants) /
total_mobility_status_5_years_ago_25_percent_sample_data)*100,
    foreign_movers = (external_migrants /
total_mobility_status_5_years_ago_25_percent_sample_data)*100
    ) %>%

  rename(AREA_SHORT_CODE = neighbourhood_number,
         young_dep = x0_to_14_years,
         old_dep = x65_years_and_over,
         median_aftertax_income =
```

```
median_after_tax_income_in_2019_among_recipients,
         gini_index = gini_index_on_adjusted_household_after_tax_income,
         with_mortgage = percent_of_owner_households_with_a_mortgage,
         mortgage_unaffordable =
percent_of_owner_households_spending_30_percent_or_more_of_its_income_on_shel
ter_costs) %>%

  dplyr::select(AREA_SHORT_CODE, area_name, young_dep, old_dep,
median_aftertax_income, gini_index, single_det_rate, semi_det_rate,
row_hse_rate, duplex_rate, eligible_type_count, unsuitability_rate,
bef60_hse_rate, x60s70s_hse_rate, x80s_hse_rate, x90s_hse_rate,
x00s_hse_rate, with_mortgage, mortgage_unaffordable, x2p_rate, x3p_rate,
x4p_rate, couples_wchildren, single_parent, vis_min_rate, domestic_movers,
foreign_movers)
```

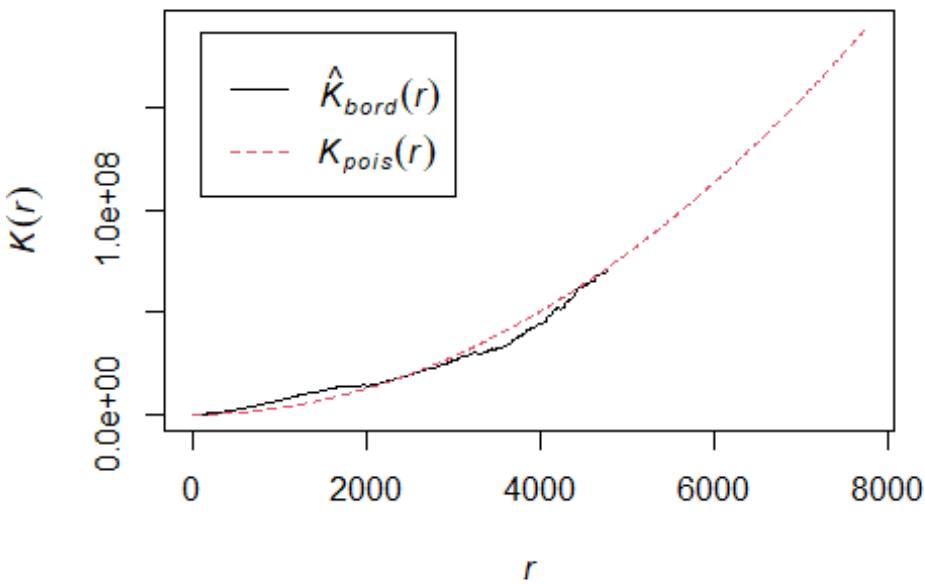
Pooling All Neighbourhood-Level Statistics Together

```
toronto_combined <- toronto_neigh %>%
  left_join(., census_cleaned, by = 'AREA_SHORT_CODE') %>%
  left_join(., cycling, by = 'AREA_SHORT_CODE') %>%
  left_join(., pt_offpeak, by = 'AREA_SHORT_CODE') %>%
  left_join(., pt_peak, by = 'AREA_SHORT_CODE') %>%
  left_join(., walking, by = 'AREA_SHORT_CODE') %>%
  mutate(
    adu_count = lengths(st_intersects(., adu_permits)),
    adu_rate = (adu_count / eligible_type_count)*100
    )
```

Doing Point Cluster Analysis

Step 1: Proving that there is clustering going on in the first place using Ripley's K

```
adu_proj <- adu_permits %>%
  st_transform(26917)

adu_sp <- adu_proj %>%
  st_cast('POINT') %>%
  as('Spatial')

toronto_proj <- toronto_neigh %>%
  st_transform(26917) %>%
  st_union()

ppp_obj <- ppp(x = adu_sp@coords[,1],
               y = adu_sp@coords[,2],
               window = as.owin(toronto_proj))

K_result <- Kest(ppp_obj, correction = "border") %>%
  plot()
```



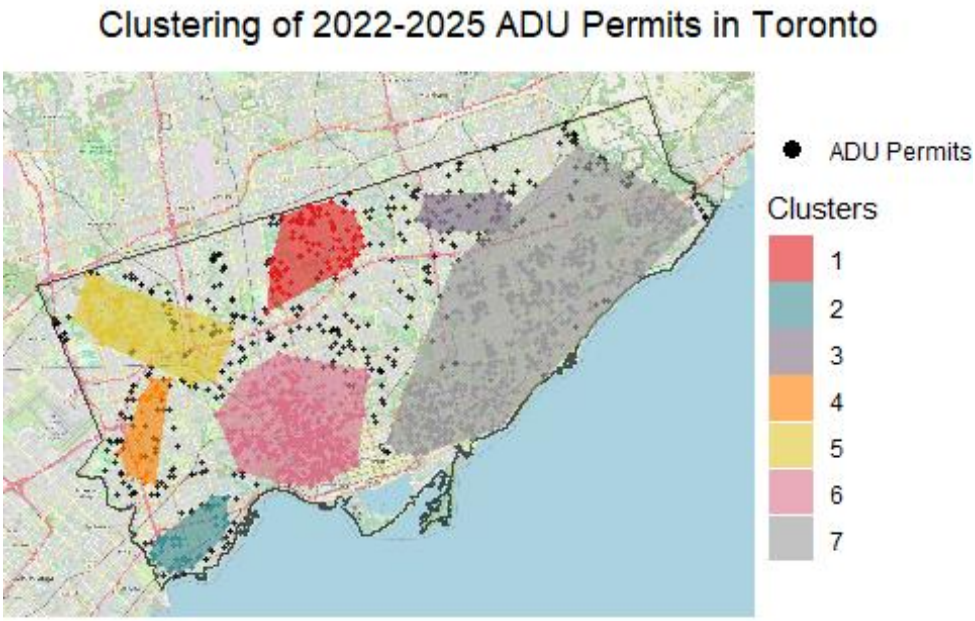
The fact that Kbord is above Kpois up until r = 2000 provides proof that there is some kind of clustering going on within ADU permit activity up until a distance of 2km.

Step 2: Showing where the point clustering is happening using HDBSCAN

The Minimum Points is set to 1% of total ADU permits, which **would be about 19**.

```
coord_df <- adu_sp %>%
  coordinates() %>%
  as.data.frame()

hdbscan <- hdbscan(coord_df, minPts = 19)

hdb_results <- coord_df %>%
  mutate(Clusters = hdbscan$cluster) %>%
  filter(Clusters != 0) %>%
  mutate(Clusters = factor(Clusters)) %>%
  st_as_sf(.,
           coords = c('coords.x1', 'coords.x2'),
           crs = 26917)

# make the hull polygons for each cluster, thus making a layer of
multipolygon spatial dataframe
cluster_polygons <- hdb_results %>%
  group_by(Clusters) %>%
  summarise(geometry = st_combine(geometry),
```

```r
            count = n()) %>%
  mutate(geometry = st_convex_hull(geometry)) %>%
  st_as_sf()

cluster_cols <- setNames(
  colorRampPalette(brewer.pal(9, "Set1"))(nlevels(hdb_results$Clusters)),
  levels(hdb_results$Clusters)
  )

# put the layers together
cluster_map <- ggplot() +

  #base maps and layers
  annotation_map_tile(zoom = 12) +
  geom_sf(data = toronto_proj, alpha = 0, lwd = 0.5) +
  geom_sf(data = adu_proj, aes(shape = 'ADU Permits'), color = 'black', alpha
= 0.8, size = 0.8) +

  #then put the cluster points
  geom_sf(data = hdb_results, aes(color = Clusters), size = 0.8, show.legend
= FALSE) +

  #then put the cluster hull polygons
  geom_sf(data = cluster_polygons, aes(fill = Clusters), alpha = 0.6, color =
NA) +

  # manually adjusting aesthetics and legends for the map
  scale_fill_manual(values = cluster_cols, name = 'Clusters') +
  scale_color_manual(values = cluster_cols, guide = 'none') +
  scale_shape_manual(values = c('ADU Permits' = 16), name = NULL) +

  labs(title = 'Clustering of 2022-2025 ADU Permits in Toronto') +

  theme_void() +

  theme(
    plot.title.position = 'plot',
    plot.title = element_text(hjust = 0.5, margin = margin(b = 10))
    ) +

  guides(
    shape = guide_legend(order = 1, override.aes = list(size = 3, alpha =
1)),
    fill = guide_legend(order = 2)
  )

cluster_map
```




```
ggsave("clustering.png", plot = cluster_map, width = 8, height = 5, dpi =
300)

# showing a table of ADU counts in each cluster
cluster_polygons

## Simple feature collection with 7 features and 2 fields
## Geometry type: POLYGON
## Dimension:     XY
## Bounding box:  xmin: 611828.2 ymin: 4827533 xmax: 650203.4 ymax: 4854289
## Projected CRS: NAD83 / UTM zone 17N
## # A tibble: 7 × 3
##   Clusters                                                        geometry
count
## * <fct>                                                       <POLYGON [m]>
<int>
## 1 1        ((623843.9 4843622, 624038.3 4846772, 624674.8 4849636, 626160…
83
## 2 2        ((618092.9 4827533, 617101.1 4828232, 616873.1 4828601, 617057…
45
## 3 3        ((634437.5 4848769, 632940.1 4849551, 633589.5 4851037, 637636…
40
## 4 4        ((616497.7 4832984, 615282.2 4833516, 614762.6 4834809, 615966…
25
## 5 5        ((619277.5 4839233, 613608.8 4841578, 611828.2 4843030, 612583…
118
## 6 6        ((626777 4832927, 625969.9 4833026, 624740 4833206, 622277.3 4…
```

```
562
## 7 7        ((633242.5 4835129, 632032 4835154, 631513.1 4835786, 631658.7…
643
```

The HDBSCAN identifies seven clusters all across Toronto, ranging from as little as 25 permits to as many as 643 permits. The notable clusters are the second-largest (which is also the densest) cluster, with 562 permits, that is located west of Downtown Toronto, stretching into High Park and Lawrence Avenue, as well as the largest cluster in size and number, with 643 permits, stretching from the Don River to Scarborough.

Now that we know where ADUs are clustering, let's see if they could be correlated with neighbourhood built or socio-demographic characteristics via regression analysis.

Conducting Principal Components Analysis

The toronto_combined object has 34 independent variables. This is NOT suitable for regression and therefore requires dimensionality reduction. Groups of relevant and related data needs to be isolated and PCA conducted on each group separately.

```
accessibility <- toronto_combined %>%
  dplyr::select(AREA_SHORT_CODE, median_cycling, median_pt_offpeak,
median_pt_peak, median_walking) %>%
  st_drop_geometry()

housing_type <- toronto_combined %>%
  dplyr::select(AREA_SHORT_CODE, single_det_rate, semi_det_rate,
row_hse_rate, duplex_rate) %>%
  st_drop_geometry()

const <- toronto_combined %>%
  dplyr::select(AREA_SHORT_CODE, bef60_hse_rate, x60s70s_hse_rate,
x80s_hse_rate, x90s_hse_rate, x00s_hse_rate) %>%
  st_drop_geometry()

family_size <- toronto_combined %>%
  dplyr::select(AREA_SHORT_CODE, x2p_rate, x3p_rate, x4p_rate) %>%
  st_drop_geometry()
```

The PCAs will be done using Python code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
```

```
sns.set(style="whitegrid", context="notebook")
plt.rcParams["figure.figsize"] = (8, 6)
```

Each PCA process will go through a standardisation BEFORE (so as to ensure that contributing factors with exceedingly high raw values do not disproportionately affect the PCs) and AFTER (to facilitate comparison with other PCs produced from other PCAs during regression analysis)

PCA for Accessibility

```
access = r.accessibility
access = access.set_index('AREA_SHORT_CODE')

scaled = StandardScaler().fit_transform(access)

# Fit and transform, then wrap back to DataFrame
access_scaled = pd.DataFrame(
    scaled,
    columns=access.columns,   # preserve column names
    index=access.index        # preserve row labels
)

pca = PCA()

# fit the components
pca_accessibility = pca.fit_transform(access_scaled)

# Explained variance ratio ------------------------------------------------
---
explained_var = pca.explained_variance_ratio_
num_components = len(explained_var)

# print explained_var and accumulative explained variance
df_explained_var = pd.DataFrame({
    "n_pc": np.arange(1, len(explained_var) + 1),
    "explained_var": explained_var,
    "cum_var": np.cumsum(explained_var)
})

print("Table of explained variance:\n", df_explained_var.round(3))
```

```
## Table of explained variance:
##     n_pc  explained_var  cum_var
## 0     1          0.880    0.880
## 1     2          0.083    0.963
## 2     3          0.035    0.998
## 3     4          0.002    1.000
```

```
# the sum of explained variance should be equal to 1.0 or 100%
print('Sum of explained variance ratio of all
PCs : {}'.format(pca.explained_variance_ratio_.sum()), '\n')

## Sum of explained variance ratio of all PCs : 1.0

# Elbow detection using "turning point" (discrete second derivative) --------
---
y = explained_var

# Need at least 3 points to compute a second derivative
if num_components < 3:
    # Fallback: choose 1 component if fewer than 3 PCs
    k = 1
else:
    # First differences: dy[i] = y[i+1] - y[i]
    dy = np.diff(y)                  # length: num_components - 1

    # Second differences: ddy[i] = dy[i+1] - dy[i]
    ddy = np.diff(dy)                # length: num_components - 2

    # Index of max absolute curvature (turning point)
    # ddy[i] is curvature at original index i+1 (0-based)
    turning_idx = np.argmax(np.abs(ddy))  # in range [0, num_components-3]

    # Convert to PC index (1-based for plotting/human interpretation)
    k = turning_idx + 2  # +1 for 0-based -> original index, +1 more because
curvature is at i+1

print(f"Chosen k based on turning-point elbow detection: {k} \n")

## Chosen k based on turning-point elbow detection: 2

# Show the loading table ----------------------------------------------------
---
df_PC = pd.DataFrame(pca.components_, columns =
access_scaled.columns.to_list()) # the list_var_X is used as the column names
print('Factor Loading Table : \n', df_PC)

## Factor Loading Table :
##     median_cycling  median_pt_offpeak  median_pt_peak  median_walking
## 0         0.479374           0.511346        0.510788        0.497817
## 1         0.660357          -0.471695       -0.476772        0.337817
## 2        -0.578029          -0.110602       -0.125240        0.798727
## 3         0.003423           0.709786       -0.704343       -0.009677
```

The turning elbow detection and explained variance suggests to choose the first two PCs because cumulatively it could explain 96% of variance across all accessibility metrics.

However, when compared with the loading table, the first PC has positive loading for every metric, which can be seen as an **general non-auto accessibility metric**. It also can explain 88% of the variance.

For ease of analysis, only the first PC will be selected and standardised for regression.

```
selected_PC = 1
pca = PCA(selected_PC, random_state=0)
scores = pca.fit_transform(access_scaled)

# Put scores in a DF with the SAME index (row_id)
pc_cols = [f"access_PC{i+1}" for i in range(selected_PC)]
df_access_pcs = pd.DataFrame(scores, index=access.index, columns=pc_cols)

# Standardise PC scores (so each has mean 0, SD 1)
scaler_pc = StandardScaler()
df_access_pcs_z = pd.DataFrame(
    scaler_pc.fit_transform(df_access_pcs),
    index=df_access_pcs.index,
    columns=[c + "_z" for c in df_access_pcs.columns]
)

# convert index back into a column
access_pcs = df_access_pcs_z.reset_index()
```

PCA for Housing Type Mix

```
housing_type = r.housing_type
housing_type = housing_type.set_index('AREA_SHORT_CODE')

scaled = StandardScaler().fit_transform(housing_type)

# Fit and transform, then wrap back to DataFrame
housing_scaled = pd.DataFrame(
    scaled,
    columns=housing_type.columns,   # preserve column names
    index=housing_type.index        # preserve row labels
)

pca = PCA()

# fit the components
pca_housing = pca.fit_transform(housing_scaled)

# Explained variance ratio ------------------------------------------------
---
explained_var = pca.explained_variance_ratio_
num_components = len(explained_var)
```

```python
# print explained_var and accumulative explained variance
df_explained_var = pd.DataFrame({
    "n_pc": np.arange(1, len(explained_var) + 1),
    "explained_var": explained_var,
    "cum_var": np.cumsum(explained_var)
})

print("Table of explained variance \n", df_explained_var.round(3))
```

```
## Table of explained variance
##     n_pc  explained_var  cum_var
## 0     1          0.352    0.352
## 1     2          0.289    0.640
## 2     3          0.223    0.864
## 3     4          0.136    1.000
```

```python
# the sum of explained variance should be equal to 1.0 or 100%
print('Sum of explained variance ratio of all
PCs : {}'.format(pca.explained_variance_ratio_.sum()), '\n')
```

```
## Sum of explained variance ratio of all PCs : 1.0
```

```python
# Elbow detection using "turning point" (discrete second derivative) --------
---
y = explained_var

# Need at least 3 points to compute a second derivative
if num_components < 3:
    # Fallback: choose 1 component if fewer than 3 PCs
    k = 1
else:
    # First differences: dy[i] = y[i+1] - y[i]
    dy = np.diff(y)                  # length: num_components - 1

    # Second differences: ddy[i] = dy[i+1] - dy[i]
    ddy = np.diff(dy)                # length: num_components - 2

    # Index of max absolute curvature (turning point)
    # ddy[i] is curvature at original index i+1 (0-based)
    turning_idx = np.argmax(np.abs(ddy))  # in range [0, num_components-3]

    # Convert to PC index (1-based for plotting/human interpretation)
    k = turning_idx + 2  # +1 for 0-based -> original index, +1 more because
curvature is at i+1

print(f"Chosen k based on turning-point elbow detection: {k} \n")
```

```
## Chosen k based on turning-point elbow detection: 3
```

```
# Show the loading table ----------------------------------------------------
df_PC = pd.DataFrame(pca.components_, columns =
housing_scaled.columns.to_list()) # the list_var_X is used as the column
names
print('Factor Loading Table : \n', df_PC)
```

```
## Factor Loading Table :
##     single_det_rate  semi_det_rate  row_hse_rate  duplex_rate
## 0          0.626417       0.127076      0.262557     0.722854
## 1         -0.434263       0.690391      0.576811     0.045448
## 2         -0.066273      -0.657228      0.744425    -0.097421
## 3         -0.643914      -0.274342     -0.210203     0.682588
```

The turning elbow detection and explained variance suggests to choose the first three PCs because they cumulatively explain up to 86% of the variance across housing type metrics.

The first PC has strong positive loading for **single-detached and duplex houses**

The second PC has strong positive loading for semi-detached and row houses while having strong negative loadings for single-detached houses. This suggests a spectrum of **semi-detached, row VS single-detached**.

Third PC has strong positive loading for row houses and strong negative loading for semi-detached houses. This suggests a spectrum of **row houses VS semi-detached**.

```
selected_PC = 3
pca = PCA(selected_PC, random_state=0)
scores = pca.fit_transform(housing_scaled)

# Put scores in a DF with the SAME index (row_id)
pc_cols = [f"housing_PC{i+1}" for i in range(selected_PC)]
df_housing_pcs = pd.DataFrame(scores, index=housing_type.index,
columns=pc_cols)

# Standardise PC scores (so each has mean 0, SD 1)
scaler_pc = StandardScaler()
df_housing_pcs_z = pd.DataFrame(
    scaler_pc.fit_transform(df_housing_pcs),
    index=df_housing_pcs.index,
    columns=[c + "_z" for c in df_housing_pcs.columns]
)

# convert index back into a column
housing_pcs = df_housing_pcs_z.reset_index()
```

PCA for Construction Period

```
const = r.const
const = const.set_index('AREA_SHORT_CODE')
```

```python
scaled = StandardScaler().fit_transform(const)

# Fit and transform, then wrap back to DataFrame
const_scaled = pd.DataFrame(
    scaled,
    columns=const.columns,   # preserve column names
    index=const.index        # preserve row labels
)

pca = PCA()

# fit the components
pca_const = pca.fit_transform(const_scaled)

# Explained variance ratio ------------------------------------------------
---
explained_var = pca.explained_variance_ratio_
num_components = len(explained_var)

# print explained_var and accumulative explained variance
df_explained_var = pd.DataFrame({
    "n_pc": np.arange(1, len(explained_var) + 1),
    "explained_var": explained_var,
    "cum_var": np.cumsum(explained_var)
})

print("Table of explained variance \n", df_explained_var.round(3))
```

```
## Table of explained variance
##     n_pc  explained_var  cum_var
## 0     1          0.449    0.449
## 1     2          0.303    0.752
## 2     3          0.155    0.907
## 3     4          0.073    0.980
## 4     5          0.020    1.000
```

```python
# the sum of explained variance should be equal to 1.0 or 100%
print('Sum of explained variance ratio of all
PCs : {}'.format(pca.explained_variance_ratio_.sum()), '\n')
```

```
## Sum of explained variance ratio of all PCs : 0.9999999999999999
```

```python
# Elbow detection using "turning point" (discrete second derivative) --------
---
y = explained_var

# Need at least 3 points to compute a second derivative
if num_components < 3:
```

```
    # Fallback: choose 1 component if fewer than 3 PCs
    k = 1
else:
    # First differences: dy[i] = y[i+1] - y[i]
    dy = np.diff(y)                  # length: num_components - 1

    # Second differences: ddy[i] = dy[i+1] - dy[i]
    ddy = np.diff(dy)                # length: num_components - 2

    # Index of max absolute curvature (turning point)
    # ddy[i] is curvature at original index i+1 (0-based)
    turning_idx = np.argmax(np.abs(ddy))  # in range [0, num_components-3]

    # Convert to PC index (1-based for plotting/human interpretation)
    k = turning_idx + 2  # +1 for 0-based -> original index, +1 more because
curvature is at i+1

print(f"Chosen k based on turning-point elbow detection: {k} \n")

## Chosen k based on turning-point elbow detection: 3

# Show the loading table ----------------------------------------------------
df_PC = pd.DataFrame(pca.components_, columns =
const_scaled.columns.to_list()) # the list_var_X is used as the column names
print('Factor Loading Table : \n', df_PC)

## Factor Loading Table :
##     bef60_hse_rate  x60s70s_hse_rate  ...  x90s_hse_rate  x00s_hse_rate
## 0         0.581330         -0.106038  ...      -0.555547      -0.316792
## 1        -0.066356          0.714001  ...      -0.074335      -0.640647
## 2         0.486915         -0.460940  ...       0.343362      -0.383610
## 3         0.193721          0.197320  ...       0.753580      -0.127885
## 4         0.618901          0.477031  ...       0.007838       0.570705
##
## [5 rows x 5 columns]
```

The turning elbow detection and explained variance suggests to choose the first three PCs. However, the difference in explained variance between the second and third PCs is almost 15%. The third PC also has mixed loading across the time periods that complicates interpretation (+ve loadings for before 60s, 80s-90s and -ve loadings for 60s-70s, 00s). I am therefore inclined to select up to **two PCs**.

The first PC has positive loading for houses built before the 60s (0.58) and negative loading for every other factor. This suggest an **before-60s dominant housing stock**.

The second PC has positive loading for houses built in the 60s-70s (0.71) and 80s (0.26), while other factors have negative loadings. This suggests a **60s-80s dominant housing stock**.

```
selected_PC = 2
pca = PCA(selected_PC, random_state=0)
scores = pca.fit_transform(const_scaled)

# Put scores in a DF with the SAME index (row_id)
pc_cols = [f"const_PC{i+1}" for i in range(selected_PC)]
df_const_pcs = pd.DataFrame(scores, index=const.index, columns=pc_cols)

# Standardise PC scores (so each has mean 0, SD 1)
scaler_pc = StandardScaler()
df_const_pcs_z = pd.DataFrame(
    scaler_pc.fit_transform(df_const_pcs),
    index=df_const_pcs.index,
    columns=[c + "_z" for c in df_const_pcs.columns]
)

# convert index back into a column
const_pcs = df_const_pcs_z.reset_index()
```

PCA for Census Family Size

```
family_size = r.family_size
family_size = family_size.set_index('AREA_SHORT_CODE')

scaled = StandardScaler().fit_transform(family_size)

# Fit and transform, then wrap back to DataFrame
family_size_scaled = pd.DataFrame(
    scaled,
    columns=family_size.columns,   # preserve column names
    index=family_size.index        # preserve row labels
)

pca = PCA()

# fit the components
pca_family_size = pca.fit_transform(family_size_scaled)

# Explained variance ratio ------------------------------------------------
---
explained_var = pca.explained_variance_ratio_
num_components = len(explained_var)

# print explained_var and accumulative explained variance
df_explained_var = pd.DataFrame({
    "n_pc": np.arange(1, len(explained_var) + 1),
    "explained_var": explained_var,
    "cum_var": np.cumsum(explained_var)
})
```

```
print("Table of explained variance \n", df_explained_var.round(3))
```

```
## Table of explained variance
##     n_pc  explained_var  cum_var
## 0     1          0.825    0.825
## 1     2          0.157    0.981
## 2     3          0.019    1.000
```

```
# the sum of explained variance should be equal to 1.0 or 100%
print('Sum of explained variance ratio of all
PCs : {}'.format(pca.explained_variance_ratio_.sum()), '\n')
```

```
## Sum of explained variance ratio of all PCs : 0.9999999999999999
```

```
# Elbow detection using "turning point" (discrete second derivative) --------
---
y = explained_var

# Need at least 3 points to compute a second derivative
if num_components < 3:
    # Fallback: choose 1 component if fewer than 3 PCs
    k = 1
else:
    # First differences: dy[i] = y[i+1] - y[i]
    dy = np.diff(y)                  # length: num_components - 1

    # Second differences: ddy[i] = dy[i+1] - dy[i]
    ddy = np.diff(dy)                # length: num_components - 2

    # Index of max absolute curvature (turning point)
    # ddy[i] is curvature at original index i+1 (0-based)
    turning_idx = np.argmax(np.abs(ddy))  # in range [0, num_components-3]

    # Convert to PC index (1-based for plotting/human interpretation)
    k = turning_idx + 2  # +1 for 0-based -> original index, +1 more because
curvature is at i+1

print(f"Chosen k based on turning-point elbow detection: {k} \n")
```

```
## Chosen k based on turning-point elbow detection: 2
```

```
# Show the loading table ----------------------------------------------------
---
df_PC = pd.DataFrame(pca.components_, columns =
family_size_scaled.columns.to_list()) # the list_var_X is used as the column
names
print('Factor Loading Table : \n', df_PC)
```

```
## Factor Loading Table :
##     x2p_rate  x3p_rate  x4p_rate
## 0  0.622138 -0.529837 -0.576383
## 1  0.141324  0.800116 -0.582960
## 2  0.770047  0.281225  0.572661
```

The turning elbow detection and explained variance suggests to choose the first two PCs because cumulatively it could explain about 98% of variance across all family size metrics.

However, when compared with the loading table, the first PC has positive loading for 2-person families and negative loading for 3-person and 4-person families, which can be seen as a **2-person VS larger families** spectrum. It also can explain 83% of the variance.

For ease of analysis, only the first PC will be selected and standardised for regression.

```
selected_PC = 1
pca = PCA(selected_PC, random_state=0)
scores = pca.fit_transform(family_size_scaled)

# Put scores in a DF with the SAME index (row_id)
pc_cols = [f"family_size_PC{i+1}" for i in range(selected_PC)]
df_family_size_pcs = pd.DataFrame(scores, index=family_size.index,
columns=pc_cols)

# Standardise PC scores (so each has mean 0, SD 1)
scaler_pc = StandardScaler()
df_family_size_pcs_z = pd.DataFrame(
    scaler_pc.fit_transform(df_family_size_pcs),
    index=df_family_size_pcs.index,
    columns=[c + "_z" for c in df_family_size_pcs.columns]
)

# convert index back into a column
family_size_pcs = df_family_size_pcs_z.reset_index()
```

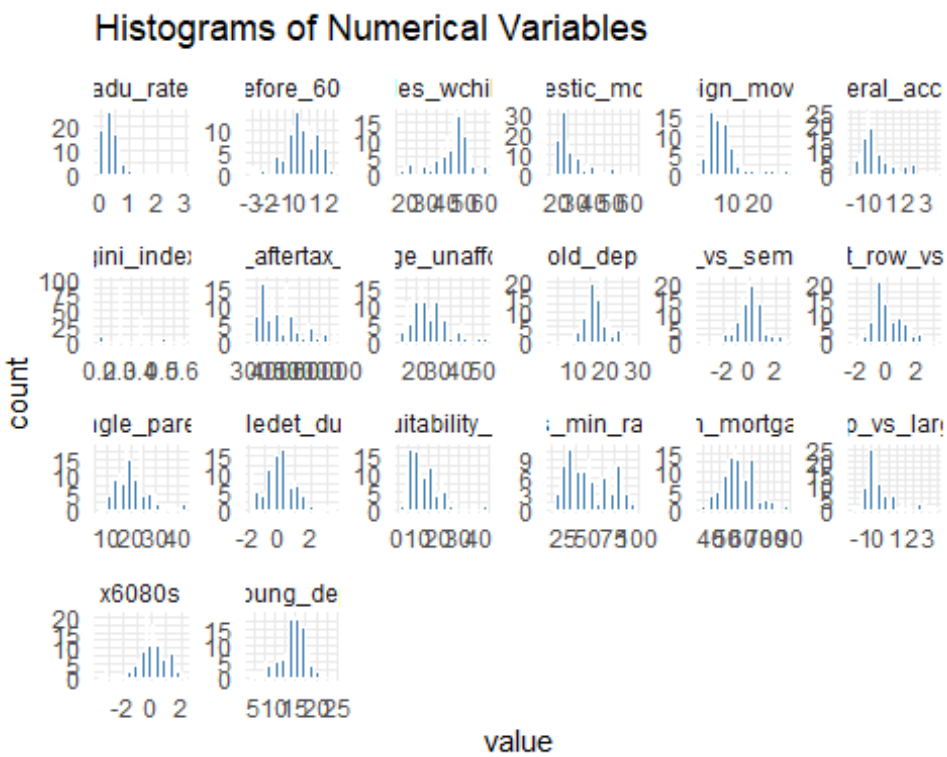
Now we need to bring the PCA, produced through Python code, back into R and merge it to form the final regressions data object.

```
access_pcs <- py$access_pcs
housing_pcs <- py$housing_pcs
const_pcs <- py$const_pcs
family_size_pcs <- py$family_size_pcs

toronto_regression <- toronto_combined %>%
  left_join(., access_pcs, by = 'AREA_SHORT_CODE') %>%
  left_join(., housing_pcs, by = 'AREA_SHORT_CODE') %>%
  left_join(., const_pcs, by = 'AREA_SHORT_CODE') %>%
  left_join(., family_size_pcs, by = 'AREA_SHORT_CODE') %>%

  rename(
```

```r
    general_access = access_PC1_z,
    singledet_duplex = housing_PC1_z,
    semidet_row_vs_single = housing_PC2_z,
    row_vs_semidet = housing_PC3_z,
    before_60s = const_PC1_z,
    x6080s = const_PC2_z,
    x2p_vs_large = family_size_PC1_z
    ) %>%

  filter(eligible_type_count != 0) %>%

  dplyr::select(AREA_SHORT_CODE, area_name, adu_rate, general_access,
singledet_duplex, semidet_row_vs_single, row_vs_semidet, before_60s, x6080s,
young_dep, old_dep, domestic_movers, foreign_movers, vis_min_rate,
x2p_vs_large, couples_wchildren, single_parent, with_mortgage,
mortgage_unaffordable, unsuitability_rate, median_aftertax_income,
gini_index)
```

Let's Do Regression

```r
# Select only numeric columns
numeric_cols <- toronto_regression %>%
  dplyr::select(-AREA_SHORT_CODE, -area_name) %>%
  st_drop_geometry()

# Convert to long format for faceting
numeric_long <- numeric_cols %>%
  pivot_longer(cols = everything(), names_to = "variable", values_to =
"value")

# Plot faceted histograms
ggplot(numeric_long, aes(x = value)) +
  geom_histogram(bins = 30, fill = "steelblue", color = "white") +
  facet_wrap(~ variable, scales = "free", ncol = 6) +
  theme_minimal() +
  labs(title = "Histograms of Numerical Variables")
```



domestic_movers, foreign_movers, vis_min_rate, single_parent, couples_wchildren, median_aftertax_income, gini_index, mortgage_unaffordable and unsuitability_rate will be log-transformed due to their non-normal distributions. Factors that came from PCAs would not be log-transformed even if the distribution is non-normal so as to facilitate regression interpretation.

adu_rate will be log-transformed via log1p(adu_rate) instead of log(adu_rate) because some neighbourhoods have no ADUs –> ADU rates of 0. In small rates, the difference between log1p(adu_rate) and log(adu_rate) is almost negligible thus can be interpreted in the same way.

Since there are 19 independent variables (out of the original 34), OLS regression will be done by adding variable chunk-by-chunk (accessibility, building stock, demographics, pressures) and getting the highest adjusted R-squared value and all significant variables in each chunk before moving on.

```
access_only <- lm(log1p(adu_rate) ~
                    general_access,
                  data = toronto_regression)

summary(access_only)

##
## Call:
## lm(formula = log1p(adu_rate) ~ general_access, data = toronto_regression)
##
```

```
## Residuals:
##      Min       1Q   Median       3Q      Max 
## -0.43511 -0.14743 -0.01845  0.11046  0.96950 
## 
## Coefficients:
##                Estimate Std. Error t value Pr(>|t|)    
## (Intercept)     0.32244    0.01813  17.789   <2e-16 ***
## general_access  0.03152    0.01902   1.657   0.0996 .  
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## 
## Residual standard error: 0.227 on 155 degrees of freedom
## Multiple R-squared:  0.0174, Adjusted R-squared:  0.01106 
## F-statistic: 2.745 on 1 and 155 DF,  p-value: 0.0996
```

General accessibility alone has a significant relationship with ADU rates

```
access_building <- lm(log1p(adu_rate) ~
                    general_access +
                    singledet_duplex +
                    semidet_row_vs_single +
                    row_vs_semidet +
                    before_60s +
                    x6080s,
                  data = toronto_regression)

summary(access_building)
```

```
## 
## Call:
## lm(formula = log1p(adu_rate) ~ general_access + singledet_duplex + 
##     semidet_row_vs_single + row_vs_semidet + before_60s + x6080s, 
##     data = toronto_regression)
## 
## Residuals:
##      Min       1Q   Median       3Q      Max 
## -0.38166 -0.11157 -0.00666  0.09162  0.78215 
## 
## Coefficients:
##                       Estimate Std. Error t value Pr(>|t|)    
## (Intercept)           0.321706   0.015322  20.997  < 2e-16 ***
## general_access        0.082630   0.021071   3.922 0.000133 ***
## singledet_duplex      0.113927   0.018475   6.167 6.17e-09 ***
## semidet_row_vs_single 0.027791   0.015734   1.766 0.079376 .  
## row_vs_semidet        0.011385   0.017713   0.643 0.521354    
## before_60s            0.061072   0.018432   3.313 0.001156 ** 
## x6080s                0.006344   0.017051   0.372 0.710367    
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## 
```

```
## Residual standard error: 0.1919 on 150 degrees of freedom
## Multiple R-squared:  0.3208, Adjusted R-squared:  0.2936
## F-statistic: 11.81 on 6 and 150 DF,  p-value: 8.019e-11
```

```
access_building2 <- lm(log1p(adu_rate) ~
                    general_access +
                    singledet_duplex +
                    semidet_row_vs_single +
                    before_60s,
                  data = toronto_regression)

summary(access_building2)
```

```
##
## Call:
## lm(formula = log1p(adu_rate) ~ general_access + singledet_duplex +
##     semidet_row_vs_single + before_60s, data = toronto_regression)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.36369 -0.11611 -0.00986  0.10001  0.80049
##
## Coefficients:
##                       Estimate Std. Error t value Pr(>|t|)
## (Intercept)            0.32173    0.01525  21.096  < 2e-16 ***
## general_access         0.07952    0.01956   4.065 7.69e-05 ***
## singledet_duplex       0.11375    0.01837   6.193 5.26e-09 ***
## semidet_row_vs_single  0.02962    0.01533   1.932 0.055192 .
## before_60s             0.05571    0.01599   3.484 0.000646 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.191 on 152 degrees of freedom
## Multiple R-squared:  0.3181, Adjusted R-squared:  0.3001
## F-statistic: 17.72 on 4 and 152 DF,  p-value: 5.835e-12
```

After adding all the factors related to housing stock characteristics, the 'single-detached/duplex' dominant and 'before-60s' dominant mixes as well as the 'semi-detached/row VS single-detached' spectrum have significant relationships with ADU rates, on top of general accessibility

```
ab_demographics <- lm(log1p(adu_rate) ~
                    general_access +
                    singledet_duplex +
                    semidet_row_vs_single +
                    before_60s +
                    old_dep +
                    young_dep +
                    log(domestic_movers) +
                    log(foreign_movers) +
```

```r
                    log(vis_min_rate),
                  data = toronto_regression)

summary(ab_demographics)
```

```
##
## Call:
## lm(formula = log1p(adu_rate) ~ general_access + singledet_duplex +
##     semidet_row_vs_single + before_60s + old_dep + young_dep +
##     log(domestic_movers) + log(foreign_movers) + log(vis_min_rate),
##     data = toronto_regression)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.40157 -0.11796 -0.00660  0.07865  0.74202
##
## Coefficients:
##                        Estimate Std. Error t value Pr(>|t|)
## (Intercept)            0.449450   0.714712   0.629  0.53042
## general_access         0.045003   0.026395   1.705  0.09031 .
## singledet_duplex       0.112780   0.022586   4.993 1.66e-06 ***
## semidet_row_vs_single  0.024279   0.017358   1.399  0.16400
## before_60s             0.098275   0.024861   3.953  0.00012 ***
## old_dep               -0.001235   0.005216  -0.237  0.81312
## young_dep             -0.018002   0.009412  -1.913  0.05775 .
## log(domestic_movers)  -0.037383   0.131517  -0.284  0.77663
## log(foreign_movers)   -0.001414   0.058862  -0.024  0.98087
## log(vis_min_rate)      0.069787   0.077718   0.898  0.37068
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.189 on 147 degrees of freedom
## Multiple R-squared:  0.3544, Adjusted R-squared:  0.3149
## F-statistic: 8.966 on 9 and 147 DF,  p-value: 1.014e-10
```

```r
ab_demographics2 <- lm(log1p(adu_rate) ~
                    general_access +
                    singledet_duplex +
                    before_60s +
                    young_dep,
                  data = toronto_regression)

summary(ab_demographics2)
```

```
##
## Call:
## lm(formula = log1p(adu_rate) ~ general_access + singledet_duplex +
##     before_60s + young_dep, data = toronto_regression)
##
## Residuals:
```

```
##      Min       1Q   Median       3Q      Max
## -0.38660 -0.11473 -0.01284  0.09694  0.78250
##
## Coefficients:
##                   Estimate Std. Error t value Pr(>|t|)
## (Intercept)       0.515555   0.089089   5.787 3.96e-08 ***
## general_access    0.048715   0.024472   1.991   0.0483 *
## singledet_duplex  0.116244   0.018317   6.346 2.41e-09 ***
## before_60s        0.075590   0.017616   4.291 3.15e-05 ***
## young_dep        -0.013864   0.006283  -2.207   0.0288 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1903 on 152 degrees of freedom
## Multiple R-squared:  0.323,  Adjusted R-squared:  0.3052
## F-statistic: 18.13 on 4 and 152 DF,  p-value: 3.409e-12
```

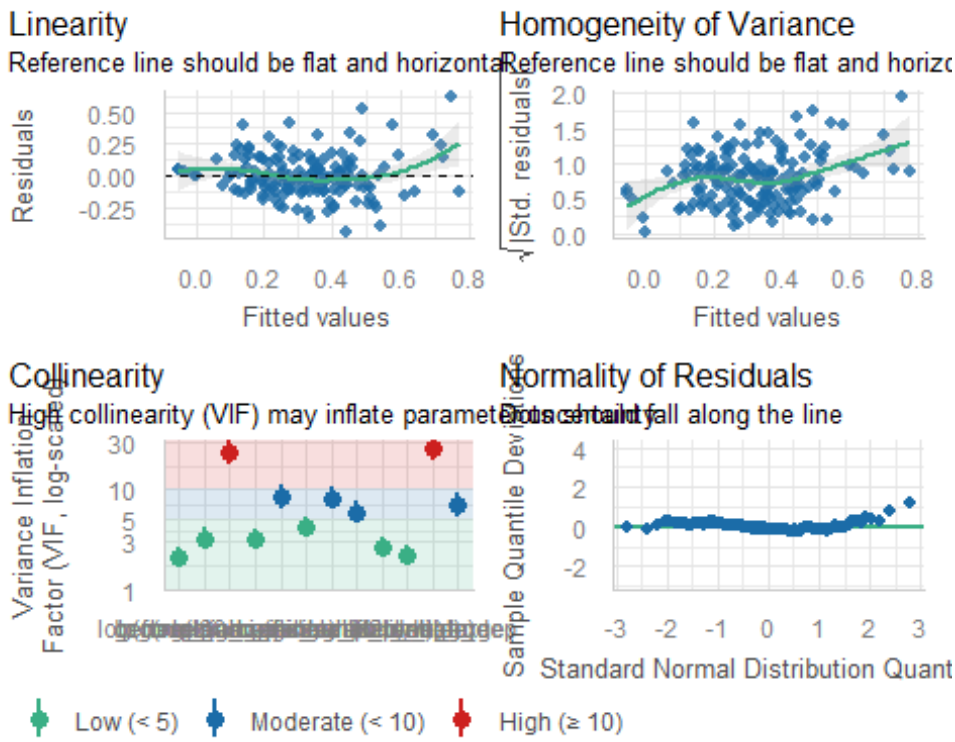
When adding the factors related to neighbourhood demographics, only the young dependent population rate has a significant relationship with ADU rates, and it only marginally improves the model. The 'semi-detached/row VS single-detached' spectrum becomes insignificant.

```r
abd_pressures <- lm(log1p(adu_rate) ~
                    general_access +
                    singledet_duplex +
                    before_60s +
                    young_dep +
                    x2p_vs_large +
                    log(couples_wchildren) +
                    log(single_parent) +
                    with_mortgage +
                    log(mortgage_unaffordable) +
                    log(unsuitability_rate) +
                    log(median_aftertax_income) +
                    log(gini_index),
                  data = toronto_regression)

summary(abd_pressures)
```

```
##
## Call:
## lm(formula = log1p(adu_rate) ~ general_access + singledet_duplex +
##     before_60s + young_dep + x2p_vs_large + log(couples_wchildren) +
##     log(single_parent) + with_mortgage + log(mortgage_unaffordable) +
##     log(unsuitability_rate) + log(median_aftertax_income) +
log(gini_index),
##     data = toronto_regression)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
```

```
## -0.44324 -0.10581 -0.01680  0.09162  0.61320
##
## Coefficients:
##                              Estimate Std. Error t value Pr(>|t|)
## (Intercept)                 -1.183691   2.756880  -0.429 0.668304
## general_access               0.067925   0.025943   2.618 0.009785 **
## singledet_duplex             0.089358   0.022384   3.992 0.000104 ***
## before_60s                   0.082344   0.019930   4.132 6.08e-05 ***
## young_dep                   -0.030654   0.010728  -2.857 0.004904 **
## x2p_vs_large                 0.204931   0.071252   2.876 0.004637 **
## log(couples_wchildren)       0.990603   0.290974   3.404 0.000859 ***
## log(single_parent)           0.416285   0.124049   3.356 0.001012 **
## with_mortgage                0.002123   0.002248   0.945 0.346407
## log(mortgage_unaffordable)  -0.062888   0.096266  -0.653 0.514620
## log(unsuitability_rate)     -0.099821   0.056029  -1.782 0.076923 .
## log(median_aftertax_income) -0.255287   0.178934  -1.427 0.155828
## log(gini_index)              0.008395   0.113243   0.074 0.941007
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1733 on 144 degrees of freedom
## Multiple R-squared:  0.468,  Adjusted R-squared:  0.4236
## F-statistic: 10.56 on 12 and 144 DF,  p-value: 8.88e-15
```

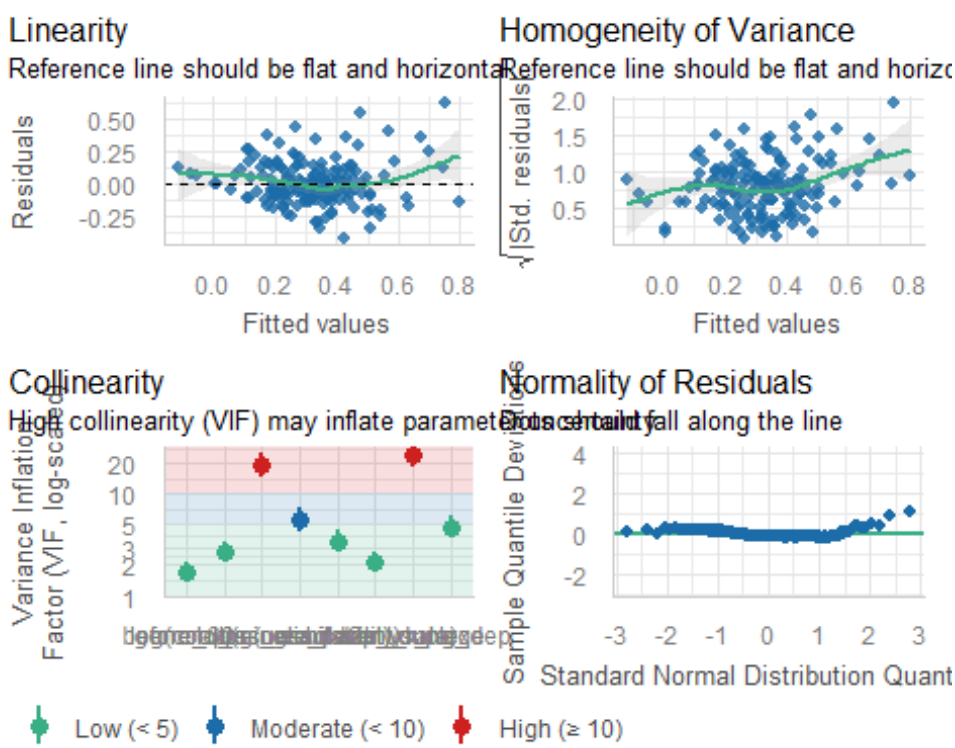
```r
# Checking for OLS assumptions of residual linearity, residual normality,
homoskedasticity, and non-collinearity of variables.
check_model(abd_pressures,
            check = c('linearity', 'homogeneity', 'qq', 'vif'))
```




```r
abd_pressures2 <- lm(log1p(adu_rate) ~
                     general_access +
                     singledet_duplex +
                     before_60s +
                     young_dep +
                     x2p_vs_large +
                     log(couples_wchildren) +
                     log(single_parent) +
                     log(unsuitability_rate),
                   data = toronto_regression)

summary(abd_pressures2)

##
## Call:
## lm(formula = log1p(adu_rate) ~ general_access + singledet_duplex +
##     before_60s + young_dep + x2p_vs_large + log(couples_wchildren) +
##     log(single_parent) + log(unsuitability_rate), data =
toronto_regression)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.42646 -0.10910 -0.01224  0.09857  0.61188
##
## Coefficients:
##                          Estimate Std. Error t value Pr(>|t|)
## (Intercept)             -4.189792   1.138444  -3.680 0.000326 ***
```

```
## general_access           0.081147   0.023736   3.419 0.000813 ***
## singledet_duplex         0.103473   0.020147   5.136 8.72e-07 ***
## before_60s               0.081248   0.017861   4.549 1.12e-05 ***
## young_dep               -0.032916   0.008773  -3.752 0.000251 ***
## x2p_vs_large             0.191920   0.068636   2.796 0.005858 **
## log(couples_wchildren)   0.975814   0.261627   3.730 0.000272 ***
## log(single_parent)       0.488508   0.101551   4.810 3.67e-06 ***
## log(unsuitability_rate) -0.048462   0.042668  -1.136 0.257875
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.173 on 148 degrees of freedom
## Multiple R-squared:  0.4555, Adjusted R-squared:  0.4261
## F-statistic: 15.48 on 8 and 148 DF,  p-value: < 2.2e-16
```

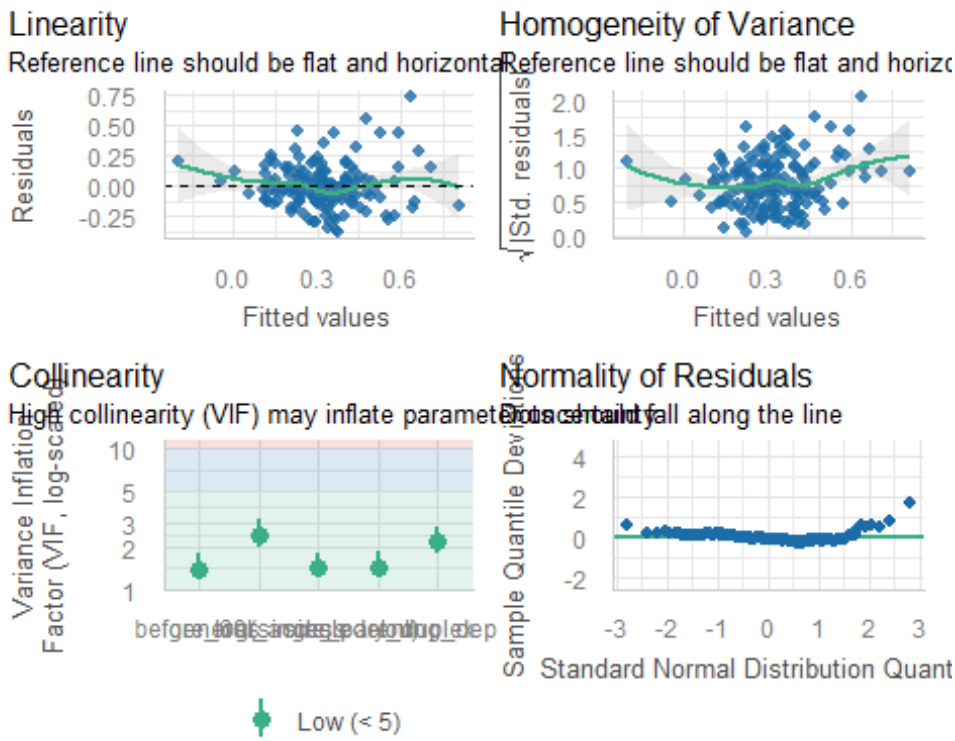
```r
# Checking for OLS assumptions of residual linearity, residual normality,
homoskedasticity, and non-collinearity of variables.
check_model(abd_pressures2,
            check = c('linearity', 'homogeneity', 'qq', 'vif'))
```



When adding all factors related to housing pressures at first, log-transformed rates of couples with children, single parents and housing unsuitability have significant relationships. However, when the model is filtered down to kick out the non-significant variables, housing unsuitability also becomes insignificant. Additionally, the 'before-60s' dominant housing stock and log-transformed rate of single parents have very high collinearity and the model is heteroskedastic. More trial-and-errors will be done in order to

get the set of predictors with the highest adjusted R-squared score but also satisfy the OLS assumptions.

```r
final_model <- lm(log1p(adu_rate) ~
                    general_access +
                    singledet_duplex +
                    before_60s +
                    young_dep +
                    log(single_parent),
                  data = toronto_regression)

summary(final_model)

##
## Call:
## lm(formula = log1p(adu_rate) ~ general_access + singledet_duplex +
##     before_60s + young_dep + log(single_parent), data =
toronto_regression)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.37606 -0.10951 -0.01094  0.08126  0.72675
##
## Coefficients:
##                     Estimate Std. Error t value Pr(>|t|)
## (Intercept)        -0.072148   0.156378  -0.461 0.645201
## general_access      0.067782   0.023475   2.887 0.004454 **
## singledet_duplex    0.121880   0.017323   7.036 6.46e-11 ***
## before_60s          0.091046   0.016973   5.364 3.00e-07 ***
## young_dep          -0.022167   0.006212  -3.568 0.000482 ***
## log(single_parent)  0.237392   0.053272   4.456 1.62e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1795 on 151 degrees of freedom
## Multiple R-squared:  0.4017, Adjusted R-squared:  0.3819
## F-statistic: 20.27 on 5 and 151 DF,  p-value: 1.911e-15

# Checking for OLS assumptions of residual linearity, residual normality,
homoskedasticity, and non-collinearity of variables.
check_model(final_model,
            check = c('linearity', 'homogeneity', 'qq', 'vif'))
```




```
# Checking for spatial autocorrelation of residuals
toronto_regression2 <- toronto_regression %>%
  mutate(residuals = residuals(final_model))

moran_i <- toronto_regression2 %>%
  pull(residuals) %>%
  as.vector() %>%
  moran.test(., nb2listw(poly2nb(toronto_regression2, queen = TRUE), style =
'W'))

moran_i

##
##  Moran I test under randomisation
##
## data:  .
## weights: nb2listw(poly2nb(toronto_regression2, queen = TRUE), style = "W")
##
## Moran I statistic standard deviate = 6.569, p-value = 2.532e-11
## alternative hypothesis: greater
## sample estimates:
## Moran I statistic       Expectation          Variance
##       0.295919164      -0.006410256       0.002118154
```

The best possible situation out of the 19 variables is to have a model (final_model) based on the following five predictors - general accessibility, a 'single-detached/duplex'

dominant housing mix, ‘before-60s’ dominant housing mix, younger-dependent population and log-transformed rate of single parents.

This is because this set of predictors achieved the highest adjusted R-squared value of 0.38, with its residuals fairly linear, slight heteroskedastic, predictors are non-collinear and a fairly normal distribution of residuals (although it slightly overpredicts at both ends of the tail).

The major thing that the OLS model is weak in is on spatial non-autocorrelation of residuals - with a Moran’s I statistic of 0.30. Lagrange multiplier tests will be carried out to decide if Spatial Lag or Spatial Error modelling should be applied.

```
lm.LMtests(final_model,
           nb2listw(poly2nb(toronto_regression, queen = TRUE), style = 'W',
zero.policy = TRUE),
           test = c("LMerr","LMlag","RLMerr","RLMlag","SARMA"))

##
##  Rao's score (a.k.a Lagrange multiplier) diagnostics for spatial
##  dependence
##
## data:
## model: lm(formula = log1p(adu_rate) ~ general_access + singledet_duplex
## + before_60s + young_dep + log(single_parent), data =
## toronto_regression)
## test weights: listw
##
## RSerr = 39.309, df = 1, p-value = 3.618e-10
##
##
##  Rao's score (a.k.a Lagrange multiplier) diagnostics for spatial
##  dependence
##
## data:
## model: lm(formula = log1p(adu_rate) ~ general_access + singledet_duplex
## + before_60s + young_dep + log(single_parent), data =
## toronto_regression)
## test weights: listw
##
## RSlag = 66.831, df = 1, p-value = 3.331e-16
##
##
##  Rao's score (a.k.a Lagrange multiplier) diagnostics for spatial
##  dependence
##
## data:
## model: lm(formula = log1p(adu_rate) ~ general_access + singledet_duplex
## + before_60s + young_dep + log(single_parent), data =
## toronto_regression)
## test weights: listw
```

```
##
## adjRSerr = 2.3439, df = 1, p-value = 0.1258
##
##
##  Rao's score (a.k.a Lagrange multiplier) diagnostics for spatial
##  dependence
##
## data:
## model: lm(formula = log1p(adu_rate) ~ general_access + singledet_duplex
## + before_60s + young_dep + log(single_parent), data =
## toronto_regression)
## test weights: listw
##
## adjRSlag = 29.866, df = 1, p-value = 4.629e-08
##
##
##  Rao's score (a.k.a Lagrange multiplier) diagnostics for spatial
##  dependence
##
## data:
## model: lm(formula = log1p(adu_rate) ~ general_access + singledet_duplex
## + before_60s + young_dep + log(single_parent), data =
## toronto_regression)
## test weights: listw
##
## SARMA = 69.175, df = 2, p-value = 9.992e-16
```

The RSlag and adjRSlag statistics are much higher with much lower p-values as compared to RSerr and adjRSerr counterparts. This suggests that a **spatial lag modelling** should be pursued.

```
lag_model <- lagsarlm(log1p(adu_rate) ~
                    general_access +
                    singledet_duplex +
                    before_60s +
                    young_dep +
                    log(single_parent),
                    data = toronto_regression,
                    nb2listw(poly2nb(toronto_regression, queen = TRUE), style
= 'W', zero.policy = TRUE),
                    method = 'eigen',
                    zero.policy = TRUE)

tidy(lag_model)
```

```
## # A tibble: 7 × 5
##   term                estimate std.error statistic       p.value
##   <chr>                  <dbl>     <dbl>     <dbl>         <dbl>
## 1 rho                   0.675    0.0650     10.4   0
## 2 (Intercept)          -0.0649   0.121      -0.537 0.591
```

```
## 3 general_access       0.0225   0.0184      1.22  0.222
## 4 singledet_duplex     0.0812   0.0137      5.94  0.00000000280
## 5 before_60s           0.0613   0.0133      4.60  0.00000427
## 6 young_dep           -0.0155   0.00479    -3.25  0.00117
## 7 log(single_parent)   0.129    0.0425      3.03  0.00241
```

```
#then you generate a sparse matrix
sparse_mat <- trW(as(nb2listw(poly2nb(toronto_regression, queen = TRUE),
style = 'W', zero.policy = TRUE), 'CsparseMatrix'),
                  type = 'mult')

#then you run the impacts function
imp <- impacts(lag_model, tr = sparse_mat, R = 2000)
summary(imp, zstats = TRUE, short = TRUE)
```

```
## Impact measures (lag, trace):
##                                Direct    Indirect       Total
## general_access dy/dx      0.02548217  0.04375192  0.06923409
## singledet_duplex dy/dx    0.09182340  0.15765730  0.24948070
## before_60s dy/dx          0.06938374  0.11912925  0.18851300
## young_dep dy/dx          -0.01758620 -0.03019484 -0.04778104
## log(single_parent) dy/dx  0.14581227  0.25035413  0.39616640
## ========================================================
## Simulation results ( variance matrix):
## ========================================================
## Simulated standard errors
##                                Direct   Indirect      Total
## general_access dy/dx     0.020774733 0.04036994 0.05971707
## singledet_duplex dy/dx   0.014844532 0.06224131 0.07052287
## before_60s dy/dx         0.015064433 0.04777438 0.05776898
## young_dep dy/dx          0.005261118 0.01449029 0.01857583
## log(single_parent) dy/dx 0.046876593 0.10720116 0.14366487
##
## Simulated z-values:
##                              Direct  Indirect     Total
## general_access dy/dx      1.274659  1.149741  1.220684
## singledet_duplex dy/dx    6.250782  2.707394  3.705205
## before_60s dy/dx          4.573913  2.613929  3.354434
## young_dep dy/dx          -3.297755 -2.184733 -2.638229
## log(single_parent) dy/dx  3.113450  2.429033  2.828409
##
## Simulated p-values:
##                          Direct     Indirect  Total
## general_access dy/dx     0.20243007 0.2502505 0.22220575
## singledet_duplex dy/dx   4.084e-10  0.0067814 0.00021122
## before_60s dy/dx         4.787e-06  0.0089508 0.00079527
## young_dep dy/dx          0.00097461 0.0289084 0.00833402
## log(single_parent) dy/dx 0.00184914 0.0151392 0.00467800
```

Using queen’s contiguity rule in our spatial lag modelling produces a rho estimate of 0.67, suggesting that the spillover effect is very strong. Additionally, general non-auto accessibility becomes insignificant.